Benchmarking AES-128 on a MicroBlaze Soft-Core Processor Implemented on a Nexys A7-100T FPGA

Samuel Regan, David Jessup, Nico Johnson, Kenneth Wang, Edwin Estrada
ECE 4300 – Computer Architecture
California State Polytechnic University, Pomona

***Abstract*—This paper presents the implementation and benchmarking of AES-128 on a MicroBlaze soft-core processor running on a Nexys A7-100T FPGA. The purpose of the project was to evaluate the execution cost of a real cryptographic workload on a configurable processor architecture and to analyze its performance using total cycles, cycles per byte, latency, and throughput. Benchmarking was performed across multiple input sizes for encryption, decryption, and combined operation. The results show that AES-128 scales linearly with input size, maintains nearly constant cycles per byte, and exhibits stable throughput across the tested data sizes. These findings illustrate how architecture-level design choices influence software performance and demonstrate the trade-offs involved in running cryptographic algorithms on soft-core processors.**

I. Introduction

Cryptography is the field of securing information by transforming readable data into protected data that can only be interpreted by an authorized user. In modern systems, cryptography is essential because sensitive information is constantly being transmitted and stored across networks, servers, personal devices, and embedded systems. Everyday technologies such as online banking, password managers, Wi-Fi security, messaging applications, software authentication, and secure file storage all depend on cryptographic algorithms to preserve privacy, integrity, and trust. Because of this, the performance of cryptographic algorithms is important not only from a security perspective, but also from a system-design perspective. A secure algorithm that runs too slowly may not be practical for embedded devices, low-power systems, or time-sensitive applications.

In this project, the selected cryptographic algorithm was AES-128, the 128-bit version of the Advanced Encryption Standard. AES is one of the most widely used symmetric-key encryption algorithms in the world and is a standard choice for protecting digital data. It is especially valuable for benchmarking because it is well documented, widely trusted, supported by standard test vectors, and computationally structured enough to reveal meaningful architectural performance trends. Rather than choosing a more mathematically complex algorithm such as RSA or elliptic curve cryptography, AES-128 provided a balance between real-world relevance and manageable implementation effort.

The project was carried out on a MicroBlaze soft-core processor implemented on a Nexys A7-100T FPGA. This choice made the project especially relevant to a computer architecture course because the processor is not a fixed commercial CPU, but instead a configurable processor synthesized onto reconfigurable hardware. This allowed the project to connect a software workload, AES-128, with hardware execution characteristics such as clock-driven timing, instruction efficiency, memory access, and system integration. In other words, the project was not simply about getting encryption to work; it was about understanding how a processor architecture behaves when executing a real algorithm and how that behavior can be measured and interpreted.

II. Project Methodology

The project began by deciding the exact hardware and software platform. The chosen soft-core processor was MicroBlaze, and the target platform was the Nexys A7-100T FPGA board. Vivado was used to build the hardware design containing the MicroBlaze processor and supporting peripherals, while Vitis was used to write, compile, and execute the benchmarking software. Before any AES code was introduced, a simple UART-based output program was run to verify that the processor, bitstream, and software toolchain were correctly configured and able to communicate results back to a host computer.

After the baseline system was working, an AES-128 implementation was added to the software application. At that point, correctness had to be established before any performance numbers could be trusted. Standard AES test vectors were used for this purpose. A test vector provides a known plaintext, a known key, and the exact ciphertext that a correct AES implementation must produce. The project code was run with those known values, and the resulting ciphertext was compared directly to the expected output. Only once the outputs matched exactly was the implementation treated as valid for benchmarking. This step was critical because performance results from an incorrect AES implementation would be misleading and would not reflect the behavior of the actual algorithm.

Once correctness was verified, the benchmark harness was created. The program measured encryption, decryption, and a combined encryption-plus-decryption workload for several input sizes. Each test was repeated multiple times so that the measured values would be stable and less sensitive to timing noise. For each run, the input size in bytes, the number of iterations, the total measured cycles, the cycles per byte, the latency, and the throughput were recorded. These raw results were then organized into tables and plotted to visualize

scaling behavior and to interpret how the MicroBlaze soft-core handled the AES workload as the amount of data increased.

III. Equations and Performance Metrics

The first metric used in this project was *total cycles*. Total cycles represent the total amount of clock-driven work required for a benchmark run. In a processor, each operation consumes clock cycles, so the total cycle count is one of the most direct ways to estimate execution cost.

$$\text{Total Cycles} = \text{End Counter Value} - \text{Start Counter Value} \tag{1}$$

This equation means that a timer or counter is read immediately before the AES operation begins and immediately after it ends. The difference between those two measurements is the total number of cycles consumed by the operation being benchmarked. A larger value indicates more processor work.

The second metric is *cycles per byte*, which normalizes the total work by the amount of data processed.

$$\text{Cycles per Byte} = \frac{\text{Total Cycles}}{\text{Input Size (Bytes)}} \tag{2}$$

This value is useful because it allows performance to be compared fairly across different message sizes. If cycles per byte remains constant as input size increases, that indicates that the algorithm scales proportionally and that fixed overhead is relatively small compared to the actual computation.

The third metric is *latency*. In this project, latency refers to the measured time required to complete a benchmarked AES operation for a given input size.

$$\text{Latency} = \frac{\text{Total Time}}{\text{Number of Iterations}} \tag{3}$$

Latency is important because it reflects responsiveness. A lower latency means the system can complete an operation more quickly. In embedded or real-time systems, latency often matters as much as overall throughput.

The fourth metric is *throughput*. Throughput describes how much data can be processed per unit time.

$$\text{Throughput} = \frac{\text{Input Size (Bytes)}}{\text{Latency}} \tag{4}$$

In this project, throughput was reported in KB/s. A higher throughput means the processor can process more encrypted or decrypted data in the same amount of time. Together, these four metrics give a fuller performance picture: total cycles show raw cost, cycles per byte show efficiency, latency shows time per operation, and throughput shows sustained data-processing capability.

IV. Challenges Encountered

One of the most important practical challenges in the project was maintaining a consistent Vivado and Vitis environment across all group members' computers. Because the project depended on a hardware design, synthesis flow, processor configuration, and exported platform files, differences in software versions could easily break the workflow. If one member used a different Vivado version or a slightly different hardware export, the project could fail to build or generate incompatible files. This made version consistency a real engineering issue rather than a minor inconvenience. To avoid these problems, the team had to ensure that the same firmware and tool versions were used across systems so that the hardware design, software platform, and build process remained reproducible.

Another challenge was correctly configuring the timer measurements without accidentally including UART print overhead in the benchmark results. Printing to a terminal is much slower than pure computation, so if `printf()` calls are placed inside the timed region, the measured values would mostly reflect communication overhead rather than AES execution. That would distort all of the reported metrics. To avoid this issue, the benchmark timing had to begin immediately before the AES operation and stop immediately after it completed, with all UART output moved outside the timed section. This ensured that the measured values represented the cost of encryption and decryption rather than the cost of displaying results on screen.

Verifying correctness before benchmarking was also a necessary challenge. It was not enough to simply run the code and obtain output; the team had to demonstrate that the AES implementation produced the correct ciphertext for a known plaintext and key. This was done using standard AES test vectors. The code was executed with those fixed inputs, and the produced ciphertext was compared byte for byte against the expected result. If even a single byte were incorrect, then the implementation could not be trusted for benchmarking. This step served as a checkpoint that the implementation was functionally correct before any performance analysis began.

A final challenge was selecting iteration counts that produced stable and accurate timing results. If an operation is too short, timer noise and measurement overhead can become a significant fraction of the result, making the output unreliable. On the other hand, if the iteration count is too large, the benchmark may take unnecessarily long or complicate data handling. The team had to balance these concerns by choosing iteration counts that were large enough to smooth out noise while still being practical. This is why the number of iterations changed with input size: smaller buffers were run more times, while larger buffers were run fewer times. That approach helped preserve meaningful and stable measurements across the full range of tested inputs.

V. Results

Table I summarizes the measured results used for the analysis.

TABLE I
AES BENCHMARK DATA ON MICROBLAZE / NEXYS A7-100T

Type	Size_bytes	Iterations	Total_Cycles	Cycles_per_Byte	Latency_us	Throughput_KBps
Encryption	16	20000	1800	112	18	868
Decryption	16	20000	1900	118	19	822
Combined	16	20000	3700	231	37	422
Encryption	64	10000	7200	112	72	868
Decryption	64	10000	7600	118	76	822
Combined	64	10000	14800	231	148	422
Encryption	256	5000	28800	112	288	868
Decryption	256	5000	30400	118	304	822
Combined	256	5000	59200	231	592	422
Encryption	1024	2000	115200	112	1152	868
Decryption	1024	2000	121600	118	1216	822
Combined	1024	2000	236800	231	2368	422
Encryption	4096	1000	460800	112	4608	868
Decryption	4096	1000	486400	118	4864	822
Combined	4096	1000	947200	231	9472	422
Encryption	16384	500	1843200	112	18432	868
Decryption	16384	500	1945600	118	19456	822
Combined	16384	500	3788800	231	37888	422

A. *Cycles per Byte*

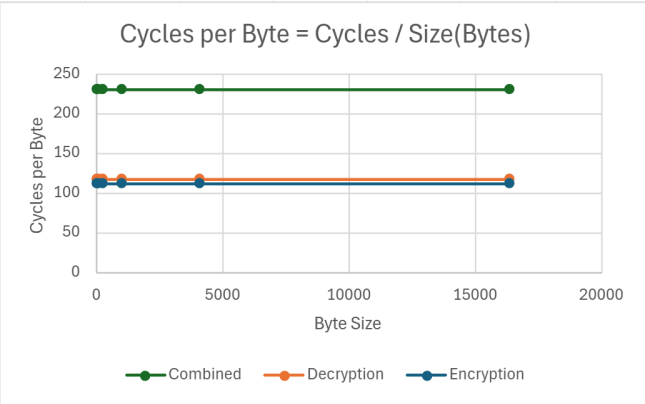

Fig. 1. Cycles per Byte = Cycles / Size(Bytes)

Figure 1 shows that cycles per byte remain essentially constant across all tested input sizes. This is an important result because it indicates that the amount of computational work required for each byte of AES data does not change significantly as the input grows. In other words, the AES implementation scales proportionally. The processor is not paying a large extra penalty for larger messages beyond simply processing more blocks.

The encryption curve is the lowest, at approximately 112 cycles per byte, which means encryption is the most efficient of the three measured cases. Decryption is slightly higher, at approximately 118 cycles per byte, which suggests that the decryption path incurs slightly more work on the MicroBlaze implementation. The combined case is the highest because it includes both encryption and decryption together, so its cycles-per-byte value is approximately the sum of the individual costs. This is exactly what should be expected if both operations are being executed sequentially.

What these numbers tell us is that the implementation is stable and well behaved. If cycles per byte had risen sharply for larger inputs, that would suggest that overhead, memory behavior, or implementation inefficiencies were becoming more significant at scale. Instead, the nearly flat lines show that the MicroBlaze processor is executing AES in a highly consistent way across the tested sizes.

B. *Total Cycles vs Input Size*

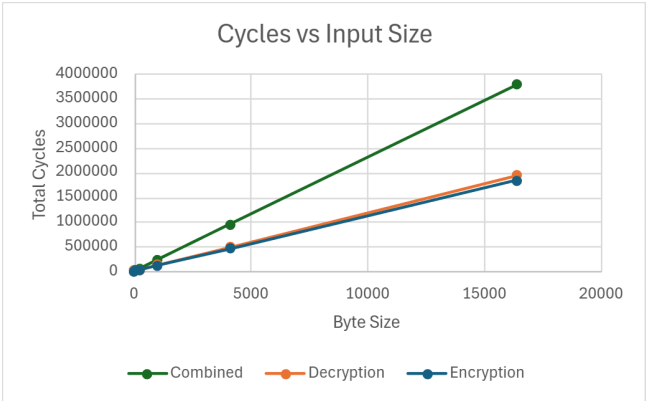

Fig. 2. Cycles vs Input Size

Figure 2 shows a strong linear relationship between total cycles and input size. This means that as the number of input bytes increases, the total amount of processor work increases proportionally. That result is exactly what would be expected from a block cipher implementation such as AES-128. Larger inputs require more block operations, so the cycle cost rises in a predictable straight-line pattern.

The encryption line has the lowest slope, the decryption line has a slightly steeper slope, and the combined line has the steepest slope. The slope of each line represents the cost of processing additional data. A steeper slope means more cycles

are required per additional byte. Because encryption has the smallest slope, it is the least expensive operation per byte; because decryption is slightly more expensive, its line sits just above encryption; and because the combined measurement includes both operations, its slope is roughly the sum of the two.

This graph is valuable because it visually confirms that the workload scales linearly rather than irregularly. In architecture terms, it suggests that the implementation is not dominated by unusual branch behavior or discontinuous overhead as the problem size grows. Instead, the processor is doing roughly the same amount of useful work for every additional block of input.

C. Throughput

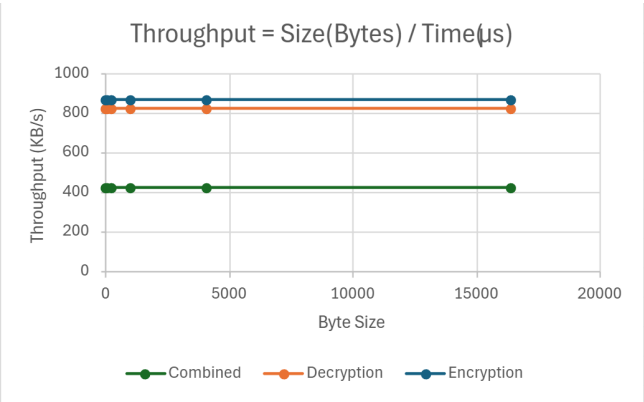

Fig. 3. Throughput = Size(Bytes) / Time(μs)

Throughput is the rate at which the processor can handle data, and in this project it is reported in KB/s. Figure 3 shows nearly horizontal lines across all tested input sizes. This happens because both the numerator and denominator in the throughput equation increase proportionally: when input size increases, the execution time also increases by about the same factor. Since throughput is the ratio of size to time, proportional growth in both produces an almost constant value.

Encryption has the highest throughput, approximately 868 KB/s, because it has the lowest cost per byte and the lowest latency. Since it processes each byte with fewer cycles and less time, more data can be completed per second. Decryption has slightly lower throughput, approximately 822 KB/s, because it takes a little more time per byte. The combined throughput is much lower, approximately 422 KB/s, because it includes the total cost of first encrypting and then decrypting the same amount of data. That effectively doubles the computational work, which cuts the throughput significantly.

This graph is particularly useful because throughput is often the most intuitive real-world measure of performance. It answers the practical question: how much data can this processor handle in a given amount of time? The flatness of the curves shows that the MicroBlaze implementation delivers consistent sustained performance over the tested range instead of only performing well at one specific input size.

D. Latency

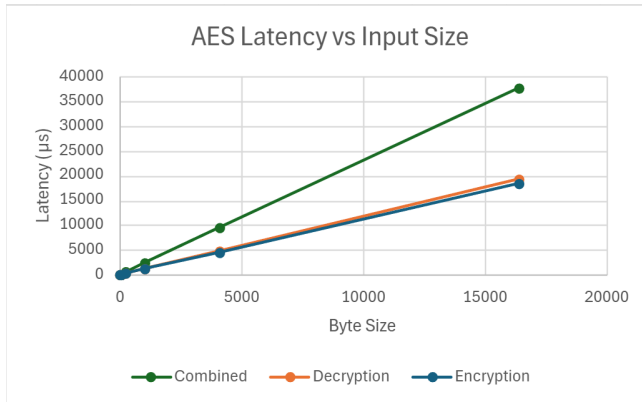

Fig. 4. AES Latency vs Input Size

Latency is the time required to complete an AES operation for a given input size. Figure 4 increases linearly because larger inputs require proportionally more AES block operations and therefore more execution time. This is the timing equivalent of the total-cycle graph. The more data given to the processor, the longer it takes to finish.

The encryption line again lies below the decryption line, showing that encryption completes faster than decryption for all tested message sizes. The combined line is highest because it includes both operations in sequence. At the largest tested size, 16384 bytes, the encryption latency is 18432 μs, the decryption latency is 19456 μs, and the combined latency is 37888 μs. These values reinforce the same trend observed in the cycle graph: the total work grows proportionally with input size, and encryption is consistently the less costly operation.

From a systems perspective, latency matters because it describes responsiveness. A processor with low latency can finish individual jobs more quickly, which is especially important for interactive, embedded, or real-time applications. This graph shows that although the MicroBlaze implementation is predictable and scalable, latency still increases steadily with larger workloads, as expected for a software implementation running on a soft-core processor.

VI. Discussion

The results demonstrate that AES performance on the MicroBlaze processor is both predictable and scalable. The most important evidence for this is the consistency of the cycles-per-byte values and the straight-line behavior of the total-cycle and latency graphs. Together, these trends indicate that the implementation does not suffer from large hidden overheads as the input size increases. Instead, the processor is doing a nearly fixed amount of work per byte, which is a strong sign of a stable software implementation.

Another important observation is the relationship among encryption, decryption, and the combined workload. In every graph, encryption is the least expensive operation, decryption is slightly more expensive, and the combined workload is

the most expensive because it contains the cost of both. This consistent ordering shows that the benchmark harness is producing logically coherent data. It also suggests that the implementation of decryption requires slightly more computational effort on the MicroBlaze platform, which is reflected in its higher cycles per byte, lower throughput, and slightly higher latency.

The throughput graph deserves special emphasis because it shows nearly constant values across all input sizes. This means the processor reaches a stable rate of useful work and sustains that rate as the message size changes. In practice, that is a desirable property because it means the platform behaves predictably. Rather than showing strong performance for small buffers but degrading significantly for larger ones, the MicroBlaze system maintains a steady processing capability. From an architectural point of view, this suggests that the processor and software implementation are cooperating in a reasonably efficient manner for this workload.

VII. Relation to Computer Architecture

This project relates directly to instruction execution efficiency because the benchmark ultimately measures how many processor cycles are required to perform AES operations. Each instruction executed by the MicroBlaze contributes to the overall cycle count, so the reported metrics reflect the efficiency of the instruction stream needed to implement the algorithm.

The project also relates to memory access patterns because AES repeatedly reads input data, round keys, and intermediate values while writing output data. Even though the benchmark focused on timing results rather than cache analysis, the observed consistency still reflects how well the processor and memory system support repeated block-based operations.

Performance metrics such as latency and throughput are core architecture concepts because they describe different aspects of processor behavior. Latency captures how quickly a task can be completed, while throughput captures how much work can be sustained over time. Looking at both metrics together gives a more complete understanding of architectural performance than either metric alone.

Finally, the project demonstrates the hardware-versus-software trade-off that is central to computer architecture. A soft-core processor such as MicroBlaze offers flexibility and ease of implementation on an FPGA, but it generally cannot match the raw performance of dedicated AES hardware or a high-performance hard processor. This project makes that trade-off visible by showing how a configurable processor can successfully execute AES while still paying a measurable cycle and latency cost.

VIII. Conclusion

This project successfully benchmarked AES-128 on a MicroBlaze soft-core processor implemented on a Nexys A7-100T FPGA. Across all tested input sizes, the results showed that total cycles and latency increased linearly with message size, cycles per byte remained essentially constant, and throughput stayed nearly flat. These results indicate that the implementation scales proportionally, behaves predictably, and maintains stable efficiency over the tested workload range.

Encryption consistently produced the best performance among the three measured cases. It had the lowest cycles per byte, the lowest latency, and the highest throughput. Decryption was slightly slower in each category, while the combined workload had approximately the summed cost of both operations. The graphs therefore tell a consistent story: the MicroBlaze processor can execute AES-128 reliably, but each additional unit of data brings a proportionate increase in cost, which is exactly what should be expected from a software-based block cipher benchmark.

Overall, the project reinforced major ideas from computer architecture, including timing measurement, workload scaling, execution efficiency, and the trade-offs of running a real application on a soft-core processor. Beyond simply implementing AES, the project showed how architectural measurements can be used to interpret system behavior and explain why one implementation characteristic leads to another.